20 PRODUCTION PROBLEMS

AWS + Java

Ecommerce

Real Production Failures

Every problem here has happened in a real ecommerce system.

Each one shows multiple solutions and trade-offs.

20	3+	100%
Real Problems	Solutions Each	Production-Grade

Swipe through all 20 problems →

Order Service Crashes During Flash Sale

SCENARIO

Black Friday sale goes live. Within 90 seconds, 50,000 users hit checkout simultaneously. ECS metrics show all tasks healthy, CPU at 40%. But RDS PostgreSQL crashes hard. The entire order pipeline goes dark.

ROOT CAUSE

Each of your 500 ECS tasks opened a HikariCP pool of 10 connections on startup. That is 5,000 simultaneous DB connections. PostgreSQL's default max_connections is 100. The database ran out of connection slots and rejected every new request.

SOLUTIONS & WHEN TO USE

● **RDS Proxy**	Best for ECS or Lambda at scale
● PgBouncer self-hosted	Cost-sensitive, team manages infra
● Reduce HikariCP pool size	Quick band-aid, not a real fix
● Aurora Serverless v2	Full auto-scaling, budget allows

SENIOR INSIGHT

RDS Proxy is zero code change and handles this transparently. PgBouncer gives finer control but you own the ops. Most teams never stress-test connection counts — Black Friday is their first real test.

Duplicate Orders Being Placed

SCENARIO

Customers start calling support. They were charged twice for a single order. Your checkout logs show the API was called exactly once per customer. Payment provider logs confirm two successful charges per transaction.

ROOT CAUSE

Your SQS visibility timeout was set to 30 seconds. Payment processing takes 35 seconds on a slow day. Before the first consumer finished, the message became visible again. A second consumer picked it up and processed the same payment a second time.

SOLUTIONS & WHEN TO USE

● **Idempotency keys on Order table**	Simplest approach, works with any queue
● SQS FIFO + deduplication ID	Queue-level guarantee, lower throughput
● Redis atomic lock SET NX EX	Fast when Redis is already in stack
● DynamoDB conditional writes	Best when already on DynamoDB

SENIOR INSIGHT

Always solve idempotency at the application layer regardless of queue type. SQS, Kafka, or RabbitMQ — all can deliver duplicates under failure. Your database is the final authority.

Orders Stuck in Processing Forever

SCENARIO

Every single day, between 50 and 100 orders get permanently stuck in a Processing state. They never move to Confirmed or Failed. Customers never receive confirmation emails. No exceptions appear anywhere in CloudWatch logs.

ROOT CAUSE

A subset of order events contain a missing or malformed productId field. Your Java consumer throws a deserialization exception, SQS retries the message 3 times, then silently routes it to the Dead Letter Queue. The DLQ has no alarm configured, so nobody ever sees it.

SOLUTIONS & WHEN TO USE

CloudWatch alarm on DLQ depth > 0	Every team. Non-negotiable baseline.
Lambda DLQ processor to Slack alert	Real-time human notification
S3 archival + manual replay tool	Audit trail and controlled reprocessing
EventBridge Pipes + Step Functions	Complex retry logic with branching

SENIOR INSIGHT

A DLQ with no alarm is a silent graveyard for real customer orders. Treat DLQ depth greater than zero as a live production incident that wakes someone up immediately.

Product Search Returns Stale Prices

SCENARIO

Your merchandising team updates a product price in the admin panel at 10:00am. At 10:20am, a customer searches for that product and sees the old price. They place the order based on the stale price. Your revenue reporting is now wrong.

ROOT CAUSE

OpenSearch is synced from RDS via a scheduled job that runs every 15 minutes. When that job fails silently due to a timeout or auth error, OpenSearch drifts from RDS indefinitely. There is no alert when the sync job fails, so the team never knows it stopped.

SOLUTIONS & WHEN TO USE

● **Transactional Outbox Pattern**	Guaranteed consistency on RDS
● DynamoDB Streams + Lambda	Already on DynamoDB
● Debezium CDC Change Data Capture	High volume, real-time, zero app code
● Dual-write with version check	Simple teams, very low write volume only

SENIOR INSIGHT

Dual-write looks simple but fails silently the moment one side succeeds and the other does not. The Transactional Outbox gives you atomicity without needing distributed transactions.

Product Images Load Slowly Worldwide

SCENARIO

Your ecommerce site is hosted in AWS us-east-1. A customer in London opens a product page. Each image takes 6 to 8 seconds to load. Customers in Singapore report even worse times. Cart abandonment in international markets is 3x higher than domestic.

ROOT CAUSE

Product images are stored in S3 us-east-1 and served directly via S3 public URLs. There is no CDN layer. Every single image request from London physically travels to Virginia and back — adding 150ms to 200ms of network latency per image, multiplied across 20 images per page.

SOLUTIONS & WHEN TO USE

● **CloudFront + S3 Origin**	Standard, tight AWS integration
○ CloudFront + S3 Transfer Accel.	When upload speed also matters
○ Imgix or Cloudinary	Need on-the-fly resizing and WebP
○ Multi-region S3 replication	Compliance requires data residency

SENIOR INSIGHT

Set Cache-Control max-age=31536000 on static product images. Only use short TTL on images that change frequently like promotional banners. Most teams apply one TTL to everything and throw away CloudFront's caching power.

Inventory Count Goes Negative

SCENARIO

A limited-edition item has exactly 1 unit left in stock. Two customers on different devices click Add to Cart at the exact same millisecond. Both orders succeed and both payments clear. Your inventory now shows minus 1. You have to cancel one order and refund a customer.

ROOT CAUSE

Your Java service reads the current stock count, checks if it is greater than zero, then decrements it. When two threads execute this read-check-write sequence at the same time, both read stock equal to 1, both pass the check, and both decrement. This is a classic check-then-act race condition.

SOLUTIONS & WHEN TO USE

● **DynamoDB conditional writes**	Atomic by design, serverless-friendly
● Redis DECR atomic command	Highest throughput for flash sales
● SELECT FOR UPDATE pessimistic lock	Simple, low concurrency on RDS only
● Optimistic locking with @Version	Moderate concurrency, fewer conflicts

SENIOR INSIGHT

Redis DECR is the fastest option for flash sales but you must reconcile the Redis counter back to your database asynchronously. Redis is your traffic controller, not your source of truth.

Payment Timeout Causes Revenue Loss

SCENARIO

Your checkout Lambda has a 30-second timeout. Your payment provider also times out after 30 seconds under heavy load. The Lambda hits its timeout limit, throws an exception, and returns a payment failure to the customer. But the payment provider already processed the charge. The customer sees an error but their card was debited.

ROOT CAUSE

Payment processing is fully synchronous inside a Lambda function. When the Lambda execution context dies, you lose the response from the payment provider. The charge has already gone through on their side but you have no record of it on yours.

SOLUTIONS & WHEN TO USE

Webhook-based async pattern	Required for all production payments
AWS Step Functions with retry	Complex multi-step payment flows
SQS + Lambda exponential backoff	Simpler than Step Functions, same safety
Increase Lambda timeout only	Never do this alone. Masks the real problem.

SENIOR INSIGHT

Payments must never be synchronous fire-and-forget. Assume the response can be lost at any point. Design for at-least-once delivery and make payment processing idempotent on the provider side too.

Refund Service Fails Silently

SCENARIO

Your customer service team starts receiving a flood of complaints. Refunds approved days ago have not been processed. You check CloudWatch — zero errors, zero exceptions, zero failed invocations. The Lambda execution count looks normal. The money simply never moved.

ROOT CAUSE

Your refund Lambda is invoked asynchronously using fire-and-forget. When it hit AWS Lambda's concurrency limit for your account, new invocations were throttled and silently dropped. Because there is no onFailure destination configured, those dropped invocations disappear with no trace.

SOLUTIONS & WHEN TO USE

● **Lambda Destinations on failure to SQS**	For all async Lambda invocations
● SQS-triggered Lambda instead	Best default for non-realtime processing
● Step Functions with error catching	Multi-step refund workflows
● EventBridge with DLQ configured	Event-driven architecture in place

SENIOR INSIGHT

Async Lambda invocations without a failure destination are one of the most dangerous patterns in AWS. The default behavior is silent discard. Always configure an onFailure destination to an SQS queue on every async Lambda.

Delivery Notifications Sent to Wrong Customers

SCENARIO

Your support inbox explodes. Customers are receiving shipment tracking updates for orders they did not place. Customer A can see Customer B's full name, address, and order contents in the notification. This is a live GDPR data breach. Your legal team is immediately involved.

ROOT CAUSE

Your notification service publishes all delivery events to a single shared SNS topic. Every customer's SQS queue subscribes to this topic without any filter policy. Every subscriber receives every message. Your Java consumer was supposed to filter by customerId but the filter logic had a bug that let all messages through.

SOLUTIONS & WHEN TO USE

● **SNS Message Filtering by customerId**	Fan-out to per-customer SQS queues
● One SNS topic per customer	Small user base only, does not scale
● EventBridge content-based routing	Complex rules, cross-account delivery
● Direct SQS per customer	High-volume, predictable customer set

SENIOR INSIGHT

Never use a shared SNS topic for customer-specific events without SNS Message Filter Policies. Filtering at the consumer is fragile. Let AWS enforce the routing at the infrastructure level — it is free and far more reliable.

Shipping Service Down, Checkout Breaks

SCENARIO

Your third-party shipping cost API goes down for 45 minutes during peak afternoon traffic. For the entire duration, every customer who tries to check out gets a 500 error. Nobody can complete a purchase. Forty-five minutes of complete checkout blackout because of one optional API call.

ROOT CAUSE

Your Java checkout service calls the shipping API synchronously and has no fallback if it fails. When the external API returns a timeout, the uncaught exception propagates all the way up and returns a 500 to the customer. Shipping cost calculation is treated as a hard dependency when it should be optional.

SOLUTIONS & WHEN TO USE

● **Circuit Breaker + flat rate fallback**	All external API calls, non-negotiable
● Cached last-known rates in ElastiCache	Shipping rates change infrequently
● Flat rate from SSM Parameter Store	Ops team can update without deploying
● Async shipping calculation post-order	Decouple UX from shipping accuracy

SENIOR INSIGHT

Shipping cost is not on the critical path for placing an order. A flat fallback rate with a reconciliation batch job running overnight is infinitely better than a completely broken checkout for 45 minutes.

Warehouse and Ecommerce Double Ship

SCENARIO

Customers start receiving two identical packages for a single order. Your warehouse management system and your Java ecommerce backend are both managing stock levels and both triggering fulfillment independently. Items are being shipped twice, costing real money on every duplicate shipment.

ROOT CAUSE

Both systems were designed to own inventory data. The ecommerce backend decrements stock on order placement. The warehouse system also decrements stock on pick-and-pack. There is no event-based coordination between them, so they both act as if they are the authority.

SOLUTIONS & WHEN TO USE

Single source of truth via EventBridge	Event-driven, clear domain ownership
Saga with compensating transactions	Complex multi-system workflows
Warehouse as master, ecommerce reads	Simple integration, low event volume
Shared inventory microservice	Greenfield, building from scratch

SENIOR INSIGHT

Two systems that both own the same data will always diverge under failure. No amount of sync jobs or reconciliation scripts fixes a broken ownership model. Assign clear data ownership at the architecture level, not the code level.

Bulk Product Import Crashes the System

SCENARIO

Your merchandising team uploads a CSV file containing 100,000 new products for a seasonal catalog refresh. They hit the import button. Within 30 seconds, the Spring Boot API pod crashes. RDS CPU hits 100% and stays there. The rest of the platform slows to a crawl while the import is running.

ROOT CAUSE

The import endpoint reads the entire CSV into memory, then inserts all 100,000 rows in a single synchronous transaction inside the HTTP request lifecycle. There is no batching, no connection throttling, and no async processing. The database is hammered with 100,000 individual INSERT statements all at once.

SOLUTIONS & WHEN TO USE

● **S3 upload + Lambda batch + SQS worker**	Standard large-scale async import
● AWS Glue ETL job	Very large datasets, complex transforms
● Step Functions + batch processing	Need progress tracking, visual workflow
● Spring Batch on ECS	Already heavy on Spring ecosystem

SENIOR INSIGHT

No bulk operation should ever run synchronously inside a user-facing API request. Return 202 Accepted immediately and process in the background. Users do not mind waiting — they mind not knowing what is happening.

AWS Credentials Leaked on GitHub

SCENARIO

A developer pushes a new feature branch to a public GitHub repository. Embedded in an application.properties file are hardcoded AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY values. GitHub's secret scanning fires an alert 4 minutes later. You now have 3 minutes before bots find those keys and start spinning up crypto miners in your AWS account.

ROOT CAUSE

The developer stored real AWS credentials directly in source code for convenience during local testing. They forgot to remove them before committing. There was no pre-commit hook, no CI secret scanning, and no IAM role configured for the service — so access keys were the only option available.

SOLUTIONS & WHEN TO USE

Deactivate key in IAM immediately	First action before deletion, to audit
CloudTrail audit last 60 minutes	Check every API call made with that key
Secrets Manager for all credentials	Permanent fix with auto-rotation built in
git-secrets pre-commit hook	Catches it before it reaches GitHub

SENIOR INSIGHT

The real long-term fix is never needing static access keys at all. ECS tasks, Lambda functions, and EC2 instances should always authenticate via IAM roles, not credentials. For CI/CD pipelines, use OIDC federation.

Customer PII Exposed in CloudWatch

SCENARIO

An external GDPR compliance audit reviews your CloudWatch log groups. The auditor finds thousands of log entries containing customer full names, email addresses, phone numbers, and the last 4 digits of payment cards — all in plain text. You are now looking at a potential regulatory fine and a mandatory data breach notification to affected customers.

ROOT CAUSE

A developer wrote log.info called with the full customer object for debugging purposes and never removed it. Java's default toString on the Customer class serializes every field, including all personally identifiable information. The logging statement went unnoticed in code review and has been running in production for 8 months.

SOLUTIONS & WHEN TO USE

CloudWatch Logs data protection policy	Immediate fix, zero code change needed
Custom toString() with masked fields	Code-level clean long-term solution
Structured JSON logging with allowlist	Best practice for all production services
Logback custom masking filter	Spring Boot teams, centralized masking

SENIOR INSIGHT

CloudWatch Logs data protection policies can automatically detect and mask PII patterns like emails, SSNs, and phone numbers across your existing log groups with zero code change. Use it as an immediate safeguard while you fix the root cause in code.

Homepage Takes 8 Seconds at Peak Hours

SCENARIO

Your Java homepage API calls six downstream microservices to assemble the page — featured products, user recommendations, active promotions, inventory status, user cart, and personalization data. At 9am it responds in 1.2 seconds. At noon peak traffic, it climbs to 8 seconds. Users bounce. Conversion drops sharply.

ROOT CAUSE

All six microservice calls are made sequentially, one after the other. Each call adds its own latency. Under load, those services respond more slowly. Six services averaging 1.3 seconds each under noon load equals 7.8 seconds total. Every millisecond of slowness in any downstream service adds directly to the user-facing response time.

SOLUTIONS & WHEN TO USE

● **CompletableFuture.allOf() parallel calls**	Immediate win, minimal code change
● ElastiCache Redis page cache 30s TTL	Same homepage content for all users
● API Gateway response caching	Zero code change, less granular control
● GraphQL aggregation layer	Long-term architecture, high investment

SENIOR INSIGHT

Combining both layers gives you the maximum benefit. CompletableFuture cuts time from the sum of all calls down to the slowest single call. ElastiCache eliminates microservice calls entirely for the vast majority of homepage traffic.

DynamoDB Costs Explode Overnight

SCENARIO

You open your AWS Cost Explorer on Monday morning. DynamoDB costs jumped from $200 last week to $4,000 overnight on Saturday. There was no traffic spike, no deployment, and no new feature launched. The bill is real. Finance is already asking questions.

ROOT CAUSE

A developer ran a one-off data migration script on Saturday afternoon that performed a full Scan operation against your largest DynamoDB table — 50 gigabytes of order history data. A Scan reads every single item in the table regardless of what you need. On a 50GB table, that consumes millions of Read Capacity Units in a single run.

SOLUTIONS & WHEN TO USE

Replace Scan with Query + partition key	Always the correct architectural fix
Add GSI for the non-key access pattern	Query by non-key attribute at scale
DynamoDB Contributor Insights	Identify exact table and operation
CloudWatch alarm on ConsumedRCU	Catch the cost spike before the bill

SENIOR INSIGHT

A single rogue Scan in a cron job or ad-hoc script can generate thousands of dollars in charges in minutes. Enable DynamoDB Contributor Insights in every production table so you can identify exactly which operation caused the spike.

One Service Failure Takes Down Everything

SCENARIO

Your product recommendation service has a memory leak that causes it to become unresponsive. Within 10 minutes of the recommendation service degrading, your entire ecommerce platform is down — homepage, search, product pages, checkout. One broken feature service took out your entire business.

ROOT CAUSE

Your homepage Java service calls the recommendation service synchronously and waits for a response. When recommendations start timing out, every homepage thread blocks waiting. Your shared thread pool fills up with stuck threads. New requests queue up and then start failing. Because the thread pool is shared across all homepage functionality, everything fails together.

SOLUTIONS & WHEN TO USE

Bulkhead with isolated thread pool	All synchronous service dependencies
Circuit Breaker with Resilience4j	Fail fast, stop waiting on dead service
Async recommendations via SQS	Fully decouple from homepage response
Make recommendations truly optional	Architecture decision, not a code fix

SENIOR INSIGHT

Recommendations are a nice-to-have feature, never a critical dependency. Any non-critical service that can take down your homepage is an architectural mistake at the design level, not an ops problem to solve at runtime.

Flash Sale Oversells Limited Products

SCENARIO

You launch a limited-edition sneaker drop. Exactly 500 pairs available. Marketing drives 10,000 users to the product page simultaneously at the moment of launch. Within 3 minutes, 1,200 orders have been confirmed and payments processed. You have sold 700 more pairs than you have in stock. Every oversold customer needs a refund and an apology.

ROOT CAUSE

Your Add to Cart flow reads the current stock count from the database, checks if it is greater than zero, then decrements it. Under 10,000 concurrent users, thousands of Java threads read stock equal to 500 at the same instant, all pass the greater-than-zero check, and all proceed to decrement. This is a classic non-atomic check-then-act race condition at massive scale.

SOLUTIONS & WHEN TO USE

Redis DECR atomic command	Highest throughput, purpose-built for this
DynamoDB conditional writes	Good if no Redis in your stack
SQS FIFO to serialize checkouts	Lower volume, can tolerate queue latency
Virtual waiting room upstream	Best user experience, limits concurrency

SENIOR INSIGHT

Redis DECR is single-threaded and atomic by design — the fastest and most reliable solution for high-concurrency inventory. Always reconcile the Redis counter back to your relational database asynchronously. Redis is your traffic cop, not your long-term source of truth.

3% Order Failure Rate, No Root Cause Found

SCENARIO

Your monitoring dashboard shows a steady 3% order failure rate. That is 300 failed orders for every 10,000 placed. You pull up CloudWatch logs across your 8 microservices — Order, Payment, Inventory, Warehouse, Notification, Shipping, Fraud, and Analytics. You have thousands of log lines but cannot connect a single failed order across all 8 services to find where it broke.

ROOT CAUSE

Each service logs independently with its own internal IDs. There is no shared correlation identifier injected at the entry point and propagated through every downstream call. You can see that something failed in the Payment service and something else failed in the Shipping service on the same day, but you cannot tell if those two failures belong to the same customer order.

SOLUTIONS & WHEN TO USE

AWS X-Ray with traceId propagation	Already on AWS, quickest to integrate
OpenTelemetry + AWS Distro ADOT	Vendor-neutral and future-proof choice
Correlation ID in SLF4J MDC	Lightweight, log-only, no new infra needed
Datadog APM or Dynatrace	Already paying for a full APM platform

SENIOR INSIGHT

Logs without a shared traceId are just expensive storage. One traceId injected at API Gateway and propagated through every SQS message header, Kafka record header, and HTTP call gives you a complete end-to-end picture of any single order.

Broken Checkout Found by Customers First

SCENARIO

Your team deploys a backend change at 5pm on a Friday. The deployment completes successfully. All ECS tasks are green. CPU is normal. Memory is normal. Error rate in CloudWatch shows zero exceptions. At 5:47pm, your support chat explodes. Customers cannot check out. You have been losing orders for 47 minutes with zero internal alerts fired.

ROOT CAUSE

Your monitoring stack only tracks infrastructure health — CPU, memory, pod restarts, and exception counts. The deployment introduced a subtle bug in the order total calculation that returns a 200 OK response but produces an incorrect cart value, causing the frontend to silently fail at the payment step. No exception is thrown, so no alarm fires.

SOLUTIONS & WHEN TO USE

● **Custom CloudWatch metric: orders/minute**	Every ecommerce team. Non-negotiable.
● CloudWatch Synthetic Canary	Catch broken checkout in under 5 minutes
● Canary deployment 10% traffic first	Prevent the problem before full rollout
● Feature flags via AWS AppConfig	Instant rollback without redeployment

SENIOR INSIGHT

Infrastructure being healthy means absolutely nothing if checkout is broken. The single most important metric in ecommerce is orders placed per minute. If that number drops below your baseline during business hours, something is wrong — regardless of what your infrastructure dashboards say.

20 Problems at a Glance

Order Management
- P1 Flash Sale Crash
- P2 Duplicate Orders
- P3 Stuck Orders

Product Catalog
- P4 Stale Prices
- P5 Slow Images
- P6 Negative Stock

Payments
- P7 Payment Timeout
- P8 Silent Refund Fail

Delivery
- P9 Wrong Notifications
- P10 Shipping Outage

Warehouse
- P11 Double Shipment
- P12 Bulk Import Crash

Security
- P13 Leaked Credentials
- P14 PII in Logs

Performance
- P15 Slow Homepage
- P16 DynamoDB Cost Spike

Resilience
- P17 Cascading Failure
- P18 Flash Sale Oversell

Observability
- P19 Mystery Failures
- P20 Checkout Down Silent

You now think like a Senior Java Engineer.

Most developers know one solution per problem.

Seniors know the trade-offs between all of them.

Design for failure first | No single right answer | Trade-offs define seniority

Save this before your next Senior interview.

Which problem have YOU faced in production?

Drop the problem number in the comments below.

Follow for more no-fluff Java Architecture content.